D&D Simulator - Simple ChatGPT Project Workflow

A one-step-at-a-time build sequence for creating the desktop application without losing scope or continuity

Project role: Implementation guide for a local, persistent, single-player text adventure using LM Studio.

CORE PRINCIPLE

The application remembers and validates. The LLM interprets, narrates, and proposes.

Purpose

Use this workflow to build the project with ChatGPT in small, reviewable steps. Complete and test one step before moving to the next. The workflow assumes the project also contains the Main instructions, Adventure Reference Guide, and LM Studio Application Architecture and Setup Guide.

Rule for every session: Tell ChatGPT the current step number. Do not ask it to build the whole application in one response. Require a working deliverable, tests, and a single clearly stated next step.

Project working method

1. **Open the project documents.** Provide or link the four authoritative documents.
2. **State the current step.** Example: “Work only on Step 3 - Database schema.”
3. **Let ChatGPT inspect before editing.** It should read the current repository structure and relevant files first.
4. **Request one complete deliverable.** A module, migration, panel, test group, or integration path that can be run.
5. **Run and verify.** Use the provided commands and report errors back with complete logs.
6. **Record completion.** Mark the step complete only when acceptance checks pass.
7. **Continue deliberately.** Ask ChatGPT to begin the next numbered step; do not skip ahead.

Step-by-step workflow

Step	Focus	Work	Done when
0	Project intake and decisions	Choose language/UI stack, repository location, operating system target, and first LM Studio model. Copy the four project documents into the working project.	A short decisions file and confirmed folder structure.

Step	Focus	Work	Done when
1	Repository and desktop skeleton	Create the repository, virtual environment, dependency file, application entry point, main window, navigation shell, configuration, logging, and tests folder.	The empty application launches and closes cleanly.
2	Campaign folders and configuration	Create per-campaign folders for database, logs, saves, exports, and visual assets. Add settings for LM Studio URL, model ID, output limits, and context budget.	A new campaign folder can be created and reopened.
3	SQLite schema and migrations	Implement tables for campaign, character, resources, skills, inventory, equipment, quests, objectives, NPCs, relationships, locations, events, summaries, visuals, and saves.	Migrations create a database and seed a test campaign.
4	Repository and service layer	Build typed data models and repository/service methods. Add transactions, stable IDs, validation, and derived-stat calculations.	Unit tests can create, read, update, and roll back state.
5	Core panels	Build Profile, Inventory, Equipment, Skills, and Quests panels. Bind them to database services; do not use the LLM for ordinary panel display.	Panel actions update the database and refresh correctly.
6	LM Studio connector	Add startup health check, model listing/selection, timeout/cancel handling, authentication option, and a basic test prompt.	The app displays connected/disconnected state and receives one response.
7	Turn result schema	Define the structured JSON response, parser, enumerated operations, and validator. Add fixtures for valid and invalid responses.	Malformed or illegal output cannot change game state.
8	Context builder	Create current scene, relevant player state, active quest, recent turn buffer, rolling summary, and memory retrieval sections with fixed token budgets.	Different action types receive different bounded context packages.
9	Turn controller	Connect player input -> context -> LM Studio -> parse -> validate -> rules -> SQLite transaction -> panel refresh -> narration.	One complete narrative turn works atomically.
10	Basic adventure play	Implement dialogue, movement, observation, checks, time advancement, item use, inventory rewards, and quest objective updates.	A short non-combat quest can be played and resumed.
11	Event log and memory	Append meaningful event records, maintain recent turns, retrieve relevant history, and refresh the Rolling Story Summary.	A long test session remains bounded and recalls relevant facts.

Step	Focus	Work	Done when
12	Quest summaries	Maintain active quest notes and generate a completion/failure summary from committed quest events. Store consequences and follow-up threads.	Completed quests leave a concise persistent record.
13	Combat and progression	Add initiative, checks, attacks, damage, healing, resources, conditions, injuries, XP, skill XP, level-ups, and combat tests.	A deterministic combat encounter completes without state drift.
14	NPCs and relationships	Add persistent NPC profiles, locations, attitudes, shared memories, hidden relationship values, and observable status.	An NPC remembers prior treatment after save/load.
15	Map, time, and world state	Add known locations, routes, travel time, weather, deadlines, NPC schedules, and persistent location changes.	Travel and deadlines update consistently.
16	Visual Registry	Implement asset folders, canonical portrait, location versions, state signatures, history, reuse, pending status, and confirmation.	Returning to a location loads the exact stored asset.
17	Save, load, repair, and replay	Add named checkpoints, backups, schema compatibility, repair reports, and event replay/checksum tests.	A campaign restores exactly and failed turns roll back.
18	Model evaluation and performance	Test candidate local models, context sizes, JSON validity, latency, token rate, memory use, and narration quality. Tune output lengths and retrieval.	A documented recommended model/configuration for the target computer.
19	Packaging and user testing	Package the desktop app, add first-run setup, diagnostics, crash-safe logging, backup/export, and a small user test campaign.	A distributable build completes the test campaign.
20	Expansion backlog	Only after the core is stable, consider mod support, embeddings, voice, advanced maps, image generation automation, multiple companions, or network access.	A prioritized backlog that does not destabilize the MVP.

Prompt to start each step

```
Read the project documents and current repository before changing anything.

Current workflow step: [STEP NUMBER AND NAME]

Complete only this step. Do not begin later steps.

Requirements:
- Preserve all existing working behavior.
- Use the database as the source of truth.
- Keep LLM output structured and validated where relevant.
- Provide complete runnable code, not isolated fragments, unless a fragment is specifically requested.
- Add or update tests for the work.
```

```
- Run the relevant tests or clearly state which commands I must run.
- Explain any assumption that affects architecture or saved data.

At the end, report:
1. What was completed.
2. Files created or changed.
3. Tests and results.
4. Any unresolved issue.
5. The single next workflow step.

Stop after this step and ask whether I want to continue.
```

Prompt for fixing a failed step

```
Stay on workflow step [NUMBER]. The previous implementation failed.

Here is the exact error or failing test output:
[PASTE COMPLETE OUTPUT]

Inspect the relevant current files before editing. Identify the root cause, make the smallest reliable
correction, and rerun or provide the exact verification commands. Do not redesign unrelated systems and do
not move to the next workflow step.
```

Completion record template

Field	Record
Step	[number and name]
Status	Not started / In progress / Blocked / Complete
Date completed	[date]
Files changed	[paths]
Tests passed	[commands and result]
Key decision	[decision and reason]
Known limitation	[limitation or none]
Next step	[single step]

Simple rules that prevent project drift

- Never build more than one workflow step at a time.
- Never let the LLM become the permanent owner of inventory, quests, statistics, or world state.
- Never parse database changes out of ordinary narration.
- Never send the full campaign transcript when a bounded snapshot and retrieval will work.
- Never call the LLM merely to display a factual panel.
- Never commit a partial turn when one required state operation fails validation.
- Never add advanced features before save/load and replay are reliable.
- Keep a changelog and migration for every saved-data schema change.
- End every ChatGPT work session with the exact next step.

First action in the new ChatGPT project

Copy and use: Upload or paste the Main instructions, Adventure Reference Guide, LM Studio Application Architecture and Setup Guide, and this workflow. Then use the prompt below.

```
Review all four project documents. Do not write application code yet.

Complete Workflow Step 0 only: create a concise project decision record that identifies the proposed language, desktop UI framework, database, LM Studio integration path, repository layout, testing approach, packaging target, and any decisions that still require my input.

Preserve the existing gameplay terminology and rules. End with the single next step and ask whether I want to continue.
```

Step 0 questions to answer

Decision	What to record
Target computer	Operating system, GPU, VRAM, RAM, and available storage.
Implementation stack	Python/PySide6 or an explicitly selected alternative.
LM Studio model	Initial model ID, quantization, loaded context length, and expected output cap.
Rules scope	Strict D&D rules, inspired/custom rules, or a hybrid.
Visual scope	Text-only MVP or planned portrait/location image integration.
Offline boundary	Localhost-only or intentional local-network access with authentication.
First milestone	A short non-combat quest that demonstrates persistence and panel updates.

Step 0 completion checklist

- All four project documents are available in the ChatGPT project.
- The technology stack and target operating system are recorded.
- The LM Studio server URL and initial model choice are recorded.
- The repository and campaign-data folder locations are decided.
- The minimum playable milestone is written in one paragraph.
- Any unresolved decision is listed before Step 1 begins.

IMPLEMENTED APPLICATION STATUS - RELEASE 25

Last reconciled: 2026-07-23

Workflow Steps 1-19 are complete. Step 20 is now the controlled post-MVP expansion boundary. IMPLEMENTATION_HISTORY.md is the detailed running record.

Canonical workflow completion

- Steps 1-14: complete through the desktop foundation, SQLite ledger, panels, LM Studio connection, structured turns, memory, quests, combat, progression, NPCs, and relationships.
- Step 15: materially complete through Phase 13 map, time, travel, camp, deadlines, routes, and persistent location state.
- Steps 16-19: complete through Releases 16-19 Visual Registry, reliability, model evaluation/performance, and packaging/first-run validation.
- Step 20: active only as a bounded expansion category; economy, crafting, UI consolidation, world-first Character Creation, the Default UI, and semantic loot/trade continuity are completed releases within it.

Post-foundation release record

Milestone	Status	Implemented result
Release 14	Complete	NPCs, relationships, inventory/map/event refinements
Release 15	Complete	Skills, equipment, item metadata, drag/drop, hover cards
Release 16	Complete	Visual Registry and Event Log/equipment refinements
Release 17	Complete	Checkpoints, repair/replay, People dialogue repair
Release 18	Complete	Model evaluation and persistent rarity presentation
Release 19	Complete	Packaging, first-run checks, explicit model loading
Release 20	Complete	Economy, bartering, merchant stock, centered controls
Release 21	Complete	Deterministic crafting and resizable People trade layout
Release 22	Complete	Skill XP bars and redundant-navigation consolidation
Release 23	Complete	World-first Character Creation and campaign generation
Release 24	Complete	Default UI, semantic icons, narrative and persistence fixes

Current next-work rule: record each approved batch as a new Release, keep its database migration number independent, and append the result to IMPLEMENTATION_HISTORY.md after verification.

Release 25 completion record

Direct semantic SVG rendering replaces atlas cropping; all 1,686 records remain tier-weighted loot candidates.

Lazy hover-card lifecycle removes top-level tooltip-window proliferation.

Model-authored narrative segments are preserved without heuristic quote rewriting.

Adventure commerce routes into People and People dialogue returns through the shared narrative transcript.

Verification: 204 automated tests, source smoke, visual previews, and packaged Release 25 smoke. SQLite schema remains 17.